Operational Foundations and Comprehensive Builtin Specification

IOSE System Model, Engineering Principles, and Complete Numeric Stack for VDR-LLM-Prolog

Registry: [HOWL-VDR-10-2026]

Series Path: [HOWL-VDR-1-2026] → [HOWL-VDR-2-2026] → [HOWL-MATH-3-2026] → [HOWL-MATH-4-2026] → [HOWL-VDR-3-2026] → [HOWL-VDR-4-2026] → [HOWL-LLM-1-2026] → [HOWL-VDR-5-2026] → [HOWL-VDR-6-2026] → [HOWL-VDR-7-2026] → [HOWL-VDR-8-2026] → [HOWL-VDR-9-2026] → [HOWL-VDR-10-2026]

DOI: 10.5281/zenodo.20225433

Date: May 2026

Domain: Applied Philosophy / Systems Architecture / Operational Engineering

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Opus 4.6.

Abstract

VDR-1 through VDR-4 built exact arithmetic and a working transformer. VDR-5 through VDR-8 built the knowledge architecture, execution layer, lifecycle, and runtime state. VDR-9 specified Orchestrated Inference — how the tools compose into multi-step reasoning processes. All nine papers specified *what* the system does. None of them specified *how to build it as an engineering system*.

This paper provides the engineering foundation. It specifies three things that the prior papers assumed but never formalized.

First, the IOSE system model. Every component in VDR-LLM-Prolog — every primitive, every KB operation, every inference notebook, the system itself — is specified as an Inputs/Outputs/Side-Effects node. Components compose into typed networks. Any component can be black-boxed at any level. The IOSE model is the blueprint from which the system is actually constructed.

Second, the operational engineering principles. Drawn from twenty-five years of production operations experience, these are not suggestions — they are Prolog facts, rules, and constraints loaded into the root KB. They govern every decision the system makes: the 90/9/0.9 priority system for tradeoffs, the knowability spectrum for evidence trust, the hearsay chain model for provenance degradation, data primacy over logic, comprehensive over aggregated design, idempotency for safe automation, and fifteen other principles that become enforceable system behavior.

Third, the comprehensive numeric builtin specification. The original 58 numeric primitives from VDR-6 are replaced by 173 builtins that expose the full mathematical capability of VDR-1 through VDR-3: exact closed and active arithmetic, lift and rebase, Q-basis transcendental operations, functional remainder series, discrete calculus, full linear algebra, probability and statistics, polynomial algebra, finite field operations, Markov chains, denominator management, integer fast paths, and bit operations. Combined with the non-numeric builtins, the system provides 448 primitives across 22 categories — every one with an IOSE declaration, comprehensively sliced from the whole.

The central claim is that a system specified in IOSE, governed by operational principles, and equipped with comprehensive exact mathematics is buildable, testable, and maintainable. The specification is the blueprint. The principles are the building code. The mathematics is the material.

---

1. Context and Purpose

1.1 What Exists

Nine papers have specified the VDR-LLM-Prolog system:

VDR-1 through VDR-4 [HOWL-VDR-4-2026] established exact fraction arithmetic. Every number is an integer triple [V, D, R] — value, denominator, remainder. The remainder slot carries exact unresolved structure that scalar systems discard. 705 tests across 23 mathematical domains produced zero VDR computation errors. The machine learning stack — softmax, autodiff, transformer forward and backward passes — operates entirely in exact fractions.

VDR-5 [HOWL-VDR-5-2026] specified Knowledge Bases as universal containers — facts, rules, constraints, scoped in a tree, with user accounts as KBs and the organizational hierarchy as the tree structure.

VDR-6 [HOWL-VDR-6-2026] specified 255 deterministic primitives invoked through compact command tokens, sandboxed operational environments, and positive credential grants.

VDR-7 [HOWL-VDR-7-2026] specified the complete model lifecycle — data sourcing through retirement — as KB operations in one tree.

VDR-8 [HOWL-VDR-8-2026] specified runtime data primitives (LRU caches, counters, locks, queues, stacks, ring buffers, bitsets), universal dotted-path addressing with integer ID acceleration, and session snapshots with disposable cloning.

VDR-9 [HOWL-VDR-9-2026] specified Orchestrated Inference — the pattern by which the LLM composes tools into multi-step investigations with four inference modes, notebooks, provenance, and confidence propagation.

1.2 What Is Missing

The nine papers describe a system with exact arithmetic, scoped knowledge, 255 primitives, lifecycle management, runtime state, and structured inference. But they do not describe how to build it as an engineering system. Three gaps remain.

No system-level interface model. The primitives have typed signatures, but there is no formal model for how components connect, how data flows between them, how side effects propagate, or how the system decomposes into verifiable subsystems. Without this, implementation is ad hoc — each developer interprets the spec differently, and the parts may not fit together.

No operational engineering principles. The system will be built, deployed, operated, and maintained by humans. The papers specify what the system does but not the engineering principles that should govern how it is built and run. Without principles, every team reinvents decisions that have known answers.

Incomplete numeric capability. VDR-6 specified 58 numeric primitives (26 arithmetic, 16 linear algebra, 16 statistics). But VDR-1 through VDR-3 demonstrated capabilities far beyond this: active arithmetic with remainder preservation, lift and rebase operations, Q-basis transcendental constants, functional remainder series, discrete calculus, polynomial algebra, finite fields, Markov chains, and denominator management. These capabilities are tested and proven but not specified as builtins. The LLM cannot access what is not specified.

1.3 What This Paper Provides

This paper fills all three gaps in one document. The three components are designed together because they depend on each other: the IOSE model defines the interfaces, the engineering principles govern how the interfaces are used, and the numeric builtins are the most complex set of interfaces to specify correctly.

---

2. The IOSE System Model

2.1 Origin

The IOSE model comes from operational engineering practice: the principle that anything can be modeled as Inputs, Outputs, and Side Effects. This is the Universal Machine concept — you black-box the internals and describe only what goes in, what comes out, and what else changes. The model has been used for twenty-five years to design, debug, and operate production systems.

2.2 The IOSE Node

Every component in VDR-LLM-Prolog is an IOSE node:

```
IOSENode = struct {

name: Text,

inputs: []TypedSlot,

outputs: []TypedSlot,

side_effects: []DeclaredEffect,

properties: []Property,

category: enum { pure, operational, composite },

logic_type: enum { operational_logic, application_logic },

description: Text,

};
```

The inputs are what the node consumes. The outputs are what it produces. The side effects are everything else it changes. The properties describe invariants (deterministic, bounded, idempotent, commutative, associative, invertible, partial). The category says whether the node is pure (no side effects), operational (side effects, requires grant), or composite (decomposes into sub-nodes). The logic type says whether the node assumes failure and handles it (operational logic) or assumes the environment works and exits on failure (application logic).

2.3 Composition

IOSE nodes compose by connecting outputs to inputs. The output type of node N must match the input type of node N+1. A command token chain is a sequence of IOSE nodes where data flows from each node's output to the next node's input, and side effects accumulate.

```
Chain = [Node1, Node2, ..., NodeN]

Chain_inputs = Node1.inputs

Chain_outputs = NodeN.outputs

Chain_side_effects = union(Node1.side_effects, ..., NodeN.side_effects)
```

Before a chain executes, the system can validate it: do the types match at every connection? Are all required grants available for operational nodes? Do the accumulated side effects violate any active constraint?

2.4 Black-Boxing

Any composite IOSE node can be viewed at two levels. From outside, it has a single IOSE interface — inputs, outputs, side effects. From inside, it decomposes into a network of sub-nodes. The composite's external side effects are the sub-network's side effects minus the internal data flows (which are outputs of one sub-node consumed as inputs by another).

The inference notebook from VDR-9 is a composite IOSE node. From outside: inputs are a problem statement and domain knowledge, outputs are a conclusion with confidence, side effects are persistent KB assertions and permanent rules. From inside: the notebook decomposes into loop phases, primitive invocations, Prolog queries, and Python executions — each an IOSE node.

2.5 IOSE Declarations for the System

Every primitive specified in VDR-6 and VDR-8 already has an implicit IOSE interface (inputs, outputs, side effects declared in its type signature). This paper makes those declarations explicit and extends them to cover all system components.

The IOSE declarations are Prolog facts in the root KB at `root.system.iose_registry`. They are always in scope. The LLM can query them to understand what a primitive does, what it requires, and what it changes. The constraint system can verify that IOSE contracts are satisfied.

Core system components:

The VDR-LLM-Prolog system as a whole is an IOSE node: inputs are user prompts, context, and active KBs; outputs are response text, KB mutations, and direct data; side effects are environment operations, grant consumption, and session state changes. It is a composite that decomposes into the KB engine, Prolog engine, primitive executor, environment manager, session manager, command token parser, and path registry — each an IOSE node with its own interface.

The KB engine takes operation type, KB path, and fact-or-query as inputs; produces query results and success status as outputs; has side effects of fact assertion, fact retraction, and mutation logging. It is operational logic — it assumes failures and handles them.

The Prolog engine takes a query and in-scope KBs as inputs; produces bindings and success-or-failure as outputs; has no side effects. It is pure and deterministic.

The primitive executor takes a primitive ID and resolved arguments as inputs; produces a result as output; has per-primitive declared side effects. It dispatches to either the pure path or the operational path based on the primitive's category.

The environment manager takes an environment ID, operation, arguments, and grant as inputs; produces an execution result and exit code as outputs; has side effects of process creation, file writing, network access, grant decrement, and execution logging. It is operational logic.

The session manager takes an operation and session name as inputs; produces session state as output; has side effects of live state capture, restore, clear, clone creation, and

clone destruction. It is operational logic.

The command token parser takes a raw token stream as input; produces parsed commands and text segments as output; has no side effects. It is pure and deterministic.

The path registry takes a dotted path as input; produces an integer ID as output; has the side effect of ID assignment if the path is new. It is operational logic with the property that IDs are stable across sessions.

2.6 IOSE Validation

With IOSE declarations for every component, the system can perform three kinds of validation:

Type compatibility. Before executing a command token chain, verify that every output type matches the next node's input type. Catch type mismatches before execution, not at runtime.

Side effect preview. Before executing a chain, collect all declared side effects. "This inference step will: mutate 3 counters, assert 2 KB facts, make 1 network request, and decrement 1 grant." The constraint system can review before execution.

Contract verification. After execution, compare declared side effects against actual logged side effects. A component that produces undeclared side effects has a contract violation. A component that fails to produce a declared output has a contract violation. These are detectable bugs, not runtime mysteries.

2.7 IOSE and the Build

The IOSE model is not just documentation. It is the blueprint. Each IOSE node becomes an implementation unit with a defined interface. Testing verifies the interface: give the declared inputs, check the declared outputs, verify the declared side effects, confirm the declared properties (determinism, boundedness, idempotency). The system is built node by node, tested node by node, and composed node by node — with the IOSE declarations as the contract that ensures the pieces fit together.

---

3. Operational Engineering Principles

3.1 Origin

These principles come from twenty-five years of building and operating production systems. They are not theoretical — they are distilled from experience. Each principle addresses a specific class of failure observed repeatedly in practice, and provides a specific countermeasure that prevents that class of failure.

The principles are encoded as Prolog facts, rules, and constraints in a KB loaded at `root.system.oso`. They are always in scope. They govern every decision the system makes — at prompt time, during inference, through the lifecycle.

3.2 Control Is the Foundation

Operations is fundamentally about control — the ability to gather information from and change an environment. Without control, no other efficiency is possible. Control requires two things: observation (you can see the state) and agency (you can change the state). A system with observation but no agency is a dashboard. A system with agency but no observation is dangerous.

The VDR-LLM-Prolog system has both. The KB provides observation — every fact, rule, constraint, data primitive, and provenance record is queryable. The primitives and command tokens provide agency — the LLM can assert facts, execute code, manage sessions, and invoke operations. Control is structural, not aspirational.

3.3 The Knowability Spectrum

Everything exists on a spectrum from fully knowable to fundamentally unknowable.

Fully knowable: A VDR computation's result. A KB fact that was read and confirmed. A pure primitive's output from known inputs. These have confidence 1/1.

Controlled system: A database query result. A Prometheus metric from instrumentation we operate. These have confidence 95-98/100 — highly reliable but subject to lag, gaps, or collection errors.

Observed external: A REST API response. A web search result. These have confidence 50-85/100 — we can read them but don't control the source.

Pattern match: An LLM-generated statement. The LLM predicted tokens based on training patterns, not computation or verification. Confidence 30/100.

Unknowable: Future state. Whether an arbitrary program halts. The user's actual intent behind an ambiguous question. Confidence 0/1 — these should not be asserted as facts.

The knowability level determines how the system treats information. Fully knowable data is trusted without verification. Controlled system data is trusted with freshness tracking. Observed external data should be cross-referenced. Pattern-matched data should trigger a search for better sources. Unknowable things should not be asserted as facts at all.

Every evidence source in the system — Prometheus, APIs, user statements, LLM generation, exact computation — has a knowability rating. The rating is a Prolog fact in the root KB. The rating feeds into the VDR-9 confidence propagation system. The rating is queryable: "why does this conclusion have confidence 72/100?" traces back through evidence sources and their knowability ratings.

3.4 The 90/9/0.9 Priority System

Each priority tier is 10x more important than the next. When two concerns compete, the one that is 10x more important wins unless the lower concern has a 10x advantage in its own domain. This is not a suggestion — it is a decision machine that produces consistent results regardless of who is deciding.

In evidence acquisition (VDR-9): Exact computation (90) over monitoring data (9) over web search (0.9) over LLM pattern matching (0.09). Always check the KB before querying Prometheus. Always query Prometheus before searching the web. Always search the web before relying on the LLM's training-time knowledge.

In lifecycle management (VDR-7): Data quality (90) over training speed (9) over checkpoint frequency (0.9). Never sacrifice data quality for faster training. Never sacrifice training completeness for more frequent checkpoints.

In prompt-time response: Correctness (90) over completeness (9) over speed (0.9) over style (0.09). A correct partial answer is better than a complete wrong one. A complete correct answer is better than a fast incomplete one.

The priority system is encoded as Prolog facts with exact VDR fractions for the weights. The inference loop from VDR-9 uses these priorities to decide which evidence source to query next, which hypothesis to test first, and when to stop gathering evidence and conclude.

3.5 Personal Experience vs Hearsay

Information you verified yourself is high-trust. Information reported by others has a trust chain that can fail at any link. Every link in the chain degrades the effective confidence.

A Prometheus metric passes through: sensor → collection agent → time series database → HTTP API → JSON parser → VDR conversion → KB fact. Six links. If each link has 99% reliability, the chain has 0.99^6 ≈ 94% effective reliability. The system tracks the chain and computes the effective confidence as the product of link confidences.

The prescription: verify personally when possible. If the LLM can check a fact by querying the KB or running a computation, it should do so rather than trusting a reported value. The verification upgrade is logged: "user stated X, system verified X via computation, confidence upgraded from 70/100 to 95/100."

3.6 Data Primacy

Data is more trustworthy, durable, and portable than logic. Data survives goal changes. Logic dies with the goals it was built for. When data and logic conflict, trust the data and fix the logic.

In the VDR-LLM-Prolog system: the KB facts are data. The Prolog rules and Python scripts are logic. The rules can be wrong — the LLM might write a bad causal rule. But the evidence facts persist. A new set of rules can be written against the same evidence. The IOSE model enforces this: inputs and outputs are data, logic stays inside the node, and nodes are replaceable as long as they satisfy their IOSE contract.

No logic should live in the data store. Stored procedures, triggers, and inline computation in the KB pollute the knowable space with unknowable behavior. The KB stores facts and rules (which are themselves data — inspectable, queryable Prolog terms). It does not execute hidden logic on assertion or retraction.

3.7 Comprehensive Over Aggregated

A comprehensive system is built top-down: define the whole, subdivide preserving total volume, no gaps, no overlaps. An aggregated system is built bottom-up: add pieces as needed, never own the whole space. Comprehensive systems trend toward consistency. Aggregated systems trend toward inconsistency.

The VDR-LLM-Prolog KB tree is comprehensive by design. It starts from root and subdivides. Every KB has a path. Every fact has a home. The builtin specification in this paper is comprehensive — we defined the whole (every operation the LLM needs), sliced it into 22 categories by operand type, and verified that the categories cover the whole space with no gaps and no overlaps.

The danger is aggregation during implementation. If modules are built independently without referencing the whole specification, they will disagree at the interfaces. The IOSE model prevents this: every module implements a declared interface, and the interfaces are validated before composition.

3.8 Idempotency

f(f(x)) = f(x). The same operation applied to the same or different starting state converges to the desired state. Idempotency makes automation safe to re-run after failure, interruption, or uncertainty about current state.

The system tags every operation as idempotent or not. Session restore is idempotent — restoring the same snapshot twice produces the same state. KB assertion of an existing fact is idempotent — no duplicate. Counter set is idempotent — setting to the same value twice is the same. Counter increment is NOT idempotent — incrementing twice produces a different result than incrementing once.

For the disposable clone pattern from VDR-8, idempotency matters: if a clone is killed and restarted, and it re-executes some KB assertions, idempotent assertions produce no duplicates. Non-idempotent operations need guards — check before executing to prevent double effects.

3.9 One Way To Do It

The production environment should have a single canonical method per task category. Not two ways to assert a fact, not three ways to query, not four ways to execute code. One canonical method. Known, documented, tested.

The builtin specification enforces this: one primitive per operation type. There is one way to sort a list (list_sort), one way to assert a fact (kb_assert), one way to execute code in an environment (env_exec). If someone finds a shortcut that bypasses the canonical method, the IOSE contract detects it — the shortcut has undeclared side effects.

3.10 Operational Logic vs Application Logic

Operational logic assumes failure, has minimal dependencies, caches locally, and is resilient. Application logic assumes the environment works and exits cleanly on failure. Both are necessary. They differ in priorities, not quality.

In the VDR-LLM-Prolog system: the KB engine, Prolog engine, primitive executor, session manager, and constraint checker are operational logic. They handle failures explicitly. The LLM's generated Python scripts are application logic. They assume the environment works. The environment manager (operational logic) wraps user scripts (application logic) and catches their failures.

The IOSE declarations tag each component with its logic type. Operational components must meet higher reliability standards: they handle partition, they minimize dependencies, they log failures. Application components are expected to fail sometimes — the operational layer handles the failure.

3.11 Models for Control vs Models for Understanding

A model for understanding is disposable, approximate, good enough for the current assessment. A model for control must be accurate and synced with reality for the resource's lifetime.

The KB tree is a model for control. It tracks actual system state — which KBs are active, what facts are asserted, what constraints hold, what data primitives contain. It must stay synced with reality. If the KB says a counter is at 7 but the actual counter is at 8, the model is broken.

The LLM's context window is a model for understanding. It approximates system state. It is good enough for the current assessment phase in the VDR-9 inference loop. If it drifts, the next assessment phase corrects it by re-reading the KB (the model for control).

The data primitives from VDR-8 bridge the two: the LLM reads counters, queues, and LRUs from the KB (control model) instead of trying to remember values from earlier turns (understanding model). The control model is the source of truth. The understanding model is a convenience.

3.12 Knowing the Present

All monitoring data is aged. You never truly know "now" — you know what was measured some time ago. The delay may be milliseconds or minutes but is never zero.

The VDR-9 evidence freshness tracking handles this: every piece of evidence has an acquisition timestamp, a data timestamp (when the external source generated the value), and a processing time (how long the pipeline took). The total age is the sum. Freshness constraints warn when evidence is too old. The system is designed for aged data, not real-time data.

3.13 Population Statistics Only

Statistics are valid for populations, not for predicting individual events. You can predict that 3 of 1000 disks will fail this month. You cannot predict which 3.

Confidence scores are population predictions: conclusions at 85/100 confidence are correct approximately 85% of the time across the population of conclusions at that confidence level. Any individual conclusion at 85/100 may be wrong. Clone drift thresholds are population-calibrated: clones that exceed 200 turns typically drift, but any individual clone might be fine at 250 or drifted at 100.

3.14 Force Multiplier Safety

Automation amplifies both fixes and failures. A bad automated process fails repeatedly, automatically, at scale. A stable operator snapshot with a subtle bug produces clones that all have the same bug.

The countermeasure is: verify before automating. The 90/9/0.9 rule applies — verification of the stable operator (90) over speed of clone deployment (9). The system enforces this through a constraint: automated processes require documented verification before activation. Overconfident conclusions (high confidence with low evidence count) are flagged.

3.15 Encoding as Prolog

All fifteen principles — and several more covering automation strategy, shortcut detection, alignment vs consistency, and conversion boundary tracking — are encoded as Prolog axioms, facts, rules, and constraints in the `root.system.oso` KB. They are loaded at system startup. They are always in scope. They are enforceable: the constraint system checks them, the inference loop respects them, and violations are logged.

The encoding includes 15 axioms (non-negotiable foundations), approximately 80 facts (knowability levels, source mappings, IOSE declarations, canonical methods, classifications, priorities), approximately 60 rules (decision procedures, validation checks, cascade logic, conflict resolution), and 21 enforceable constraints.

---

4. The Number Type Hierarchy

4.1 Why Multiple Types

The system needs exact fractions for computation, integers for counting, decimal display for human output, Q-basis compression for transcendental constants, and functional remainders for convergent series. These are not competing representations — they are layers, each serving a specific purpose, with declared conversion boundaries between them.

4.2 Five Numeric Types

VDR fraction. The primary type. Every number is a triple [V, D, R] — value, denominator, remainder. When R=0, the object is a closed rational V/D. When R≠0, the object carries exact unresolved structure. This is the ground truth. All other types convert to and from this.

Integer. Equivalent to VDR fraction [N, 1, 0]. Used for counting, indexing, IDs, bit operations, modular arithmetic. No denominator overhead when the denominator is always 1. Every integer is a valid VDR fraction — promotion is free and lossless.

Decimal display. A string representation of a fraction at declared precision. NOT a numeric type for computation. A display format only. Generated by `fraction_to_decimal(frac, digits)`. The generating fraction is always available. The decimal is a lossy view, not a value.

Q-basis compressed. A single arbitrary-precision integer over a shared power-of-two denominator: [p, 2^k, 0]. From MATH-4: 22 fundamental constants (π , e, ln2, $\sqrt{2}$, $\zeta(3)$, and 17 others) represented as integers over the shared denominator 2^335, verified at 100 digits against mpmath. Addition of two Q-basis values with the same exponent is one integer addition. Multiplication produces a result in Q(2k) requiring reprojection to Q(k) with bounded error. The Q335 rounding error is 10^66 times smaller than the Planck length.

Functional remainder. A Python callable f(depth) → VDR. Produces an exact rational at each depth via convergent series or Newton iteration. Not evaluated until explicitly resolved via `fn_resolve`. Each depth is a complete exact value, not an approximation of a limit. Square root of 2 via Newton-Raphson at depth 7 produces approximately 150 correct digits. The series for exp, log, sin, cos produce exact rationals at every term count.

4.3 Automatic Type Dispatch

The system automatically selects the computation path based on operand types. Two integers use the fast integer path — no denominator arithmetic. An integer and a fraction promotes the integer to [N, 1, 0] and uses VDR arithmetic. Two Q-basis values with the same exponent use integer addition. The dispatch is transparent to the LLM — it issues a command token for "add these two numbers" and the system picks the fastest exact path.

4.4 Conversion Boundaries

Every conversion between types has a declared boundary recording source type, target type, method, and exact error bound. Lossless conversions (integer to fraction, Q-basis to fraction) have error 0. Lossy conversions (fraction to decimal display, fraction to Q-basis) have a bounded error recorded as an exact VDR fraction.

The conversion boundary is the point where external approximate data enters the exact system. When a Prometheus metric (a decimal string) is parsed into a VDR fraction via `vdr_from_decimal_string`, the conversion is exact for terminating decimals

(0.5 → [1, 2, 0], no error) and declared for non-terminating representations. The boundary is logged as a provenance fact. The VDR-9 inference provenance chain includes every conversion boundary in the evidence path.

---

5. VDR Exact Arithmetic Builtins

5.1 Closed Arithmetic

The foundation: exact rational arithmetic on closed objects (R=0). Addition, subtraction, multiplication, division, negation, absolute value, integer power, reciprocal. Every operation produces an exact result. The closed subclass is arithmetically closed under all four operations (excluding division by zero). The invariants — commutativity, associativity, distributivity, additive identity, multiplicative identity, additive inverse, multiplicative inverse — are all exact. Not approximately satisfied. Exactly satisfied. Testable. Tested.

5.2 Active Arithmetic

Operations on objects with non-zero remainder. Same-denominator active addition sums values and combines remainder structures. Different-denominator active addition uses the lift operator to transport remainders into a shared frame. Active multiplication captures three cross-terms ($V_1 \cdot R_2$, $V_2 \cdot R_1$, $R_1 \cdot R_2$ projected) in the remainder. Active division by a closed divisor preserves full remainder structure. Active division by an active divisor is the known v1 compromise — the divisor's remainder structure is projected to an exact rational and lost. This is a declared limitation, not a hidden defect.

5.3 Lift and Rebase

Lift is the remainder transport operator. When a denominator frame changes by factor k, lift rescales the remainder to preserve exact value. Lift of an atomic remainder r by k gives kr. Lift of a composite remainder distributes over the base and all children. Lift of a child VDR [V, D, R] by k gives [kV, D, lift(R, k)] — the child's denominator is preserved, only V and R are scaled. Lift composes multiplicatively: lift(lift(R, a), b) = lift(R, a·b).

Rebase changes the top-level denominator of a VDR object. Closed rebase produces a closed result when V·B/D is an integer. Active rebase produces a mismatch witness [S, D, 0] that captures the exact part the target denominator could not absorb. Rebase preserves value equality but not structural equality.

5.4 Comparison and Ordering

Exact comparison via cross-multiplication. No epsilon. No tolerance. Two closed fractions V_1/D_1 and V_2/D_2 compare by checking $V_1 \cdot D_2$ against $V_2 \cdot D_1$ — an integer comparison with no rounding. For active objects, both are projected via the scalar projection Π and compared as closed rationals. The comparison is total and exact.

The comparison builtins include: compare (returns less/equal/greater), equal, less-than, less-or-equal, min, max, sign, is-zero, is-positive, is-negative. Every one is exact.

5.5 Rounding and Extraction

Floor, ceiling, round, and truncate produce exact integers from exact fractions. Numerator and denominator extraction return the components after normalization. State queries (is-integer, is-closed, is-active) report the object's structure. Simplification is full normalization — deterministic, idempotent.

Denominator complexity returns a triple (distinct denominators, sum of denominators, count of denominator nodes) for tracking growth during training.

---

6. Number Theory Builtins

GCD, LCM, modular remainder, exact integer division, modular exponentiation, modular inverse, extended GCD, primality testing, factorial, binomial coefficient, Fibonacci (via matrix power for large n), Euler's totient, and Chinese Remainder Theorem.

These are the foundation for the cryptographic primitives tested in VDR-2's Gym 12 (RSA encrypt/decrypt roundtrip exact), the combinatorics tested in Gym 6 (C(20,10) = 184756, B(12) = -691/2730), and the number theory tested in Gym 1 (H_{100} exact with ~85-digit denominator, $\varphi(100) = 40$).

Every operation is exact integer arithmetic — no fractions needed, no denominator overhead.

---

7. Q-Basis Transcendental Operations

From MATH-4: 22 fundamental constants stored as integers over the shared denominator 2^335. Pi plus e is one integer addition. Pi times e produces a result in Q670 requiring reprojection to Q335 with bounded error.

The builtins: Q-basis addition, subtraction, multiplication (with exact error bound), scalar multiplication by exact rational, conversion to VDR fraction, named constant retrieval, and precision bit query. The Q-basis multiplication builtin returns both the reprojected result and the exact error bound — the system always knows how much precision was lost and records it.

---

8. Functional Remainder Operations

Square root via Newton-Raphson (quadratic convergence — correct digits double per step). Exponential, logarithm, sine, cosine via truncated Taylor series (every partial sum

is an exact rational). Resolution at declared depth. Creation of custom Newton and series functional remainders.

These enable the exact softmax from VDR-4 (truncated Taylor exp producing exact rationals that sum to exactly 1) and the transcendental function evaluations tested in VDR-3's Gym 23 (K(1/2) via ${}_2F_1$ series at 500 terms producing approximately 300 correct digits).

---

9. Discrete Calculus Builtins

From VDR-1: the discrete derivative D_h f(x) = (f(x+h) - f(x)) / h is exact for any VDR step size h. The discrete derivative of x^2 at x=3 with h=1/1000 is exactly 6001/1000 — not a float near 6, but the exact rational showing the discretization term as an algebraic fact.

Left Riemann sum, trapezoidal rule, Richardson extrapolation, finite difference tables — all exact. The finite difference table detects polynomial degree exactly: Δ^n of a degree-n polynomial equals n! times the leading coefficient, and Δ^{n+1} equals zero. No float noise floor.

---

10. Linear Algebra Builtins

From VDR-1: exact vector and matrix operations. Vector addition, subtraction, scaling, dot product, squared norm, negation. Matrix addition, multiplication, scaling, transpose, matrix-vector product. Determinant via cofactor expansion. Inverse via adjugate method — M * inv(M) = I exactly, with zero off-diagonal residue. Solve via Cramer's rule. Rank via Gaussian elimination. Identity matrix, trace, matrix power, Gram-Schmidt orthogonalization.

The Hilbert matrix demonstrations from VDR-2: H_3 through H_5 inversion exact where float fails (float residual $\sim 10^{-9}$ for 5×5, VDR residual exactly 0). The 40-operation matrix roundtrip from VDR-1: zero drift.

Matrix power enables Fibonacci computation for large n, controllability matrix computation from VDR-3's Gym 21, and spectral analysis via adjacency matrix powers from Gym 16.

---

11. Probability and Statistics Builtins

Exact mean, variance, median, percentile, mode. Exact probability normalization (output sums to exactly 1), validation (all non-negative and sum exactly 1), Bayes' theorem, expected value, CDF. Joint probability tables, marginalization, conditional distributions. Entropy terms via rational log approximation.

Exact softmax via truncated Taylor exp — for logits [1,2,3] at depth 12, the outputs are 64826368/720042809, 176214841/720042809, and 479001600/720042809, summing to 720042809/720042809 = 1. The rational surrogate softmax using square-shift kernel — for the same logits with shift c=4, outputs are 4/29, 9/29, 16/29, summing to 29/29 = 1. Faster, no transcendentals, preserves ordering.

These are the probability operations tested across VDR-2's Gym 11 (binomial PMF sums to exactly 1, Markov steady state [2/3, 1/3] exact, gambler's ruin probabilities exact) and used in VDR-4's transformer (attention weights summing to exactly 1 per row).

---

12. Domain-Specific Mathematics

12.1 Polynomial Operations

Horner evaluation, polynomial addition, multiplication, division, GCD, symbolic derivative, symbolic integral, Lagrange interpolation. Tested in VDR-2's Gym 2 (interpolation through (0,1)(1,3)(2,7) recovers $1+x+x^2$ exactly) and Gym 13 (partial fraction decomposition exact, power sum $S_3(100) = S_1(100)^2 = 25502500$).

12.2 Finite Field Operations

Addition, multiplication, inverse, and power in GF(p). Tested in VDR-3's Gym 18 (all 6 GF(7) inverses verified, Hamming(7,4) encode/decode/correct all 16 codewords).

12.3 Markov Chain Operations

Exact steady-state via Cramer's rule (sum exactly 1), single-step and n-step evolution via matrix power. Tested in VDR-2's Gym 11 (steady state [2/3, 1/3] exact) and VDR-3's Gym 21 (discrete-time control system evolution exact through 5 steps).

12.4 Graph Mathematics

Adjacency matrix power for walk counting, exact PageRank via linear system solve (scores sum to exactly 1). Tested in VDR-3's Gym 16 ($A^2[0,0] = 49/36$, Dijkstra shortest path 13/12 exact).

---

13. Integer Fast Path and Bit Operations

For operations where denominators are always 1: integer addition, subtraction, multiplication, division, modular remainder, power, absolute value, sign, min, max, clamp, range generation. These are performance paths — semantically identical to VDR operations on [N, 1, 0] but avoiding denominator arithmetic.

Bit operations: AND, OR, XOR, NOT, shift left, shift right, popcount, bit width. Used for denominator complexity tracking (bit width of denominators), Q-basis exponent management, and hash computation.

---

14. Denominator Management

Denominators grow through operations. The system provides tools for monitoring and controlled reprojection:

Denominator bits and digits — track the size of the denominator for growth monitoring.

Q-basis reprojection — reproject a value to Q-basis at a declared exponent K. Returns both the reprojected value and the exact error bound (bounded by 2^(-K-1)). The reprojection is declared, logged, and bounded. This is the antidote to float's silent truncation — controlled precision reduction with exact error tracking.

Budget check — test whether denominator bit width exceeds a declared budget. Used in the training loop from VDR-7 to trigger reprojection at checkpoint boundaries.

Precision state — full report of denominator bits, denominator digits, active/closed status, remainder depth, tree node count. The precision state that float cannot provide.

---

15. Comprehensive Builtin Specification

15.1 The Slicing Method

The complete builtin set is specified by the Slicing the Pie method (OSO C16): define the whole (every operation the LLM needs that must be exact), slice into categories by operand type, slice each category into specific operations, verify no gaps, verify no overlaps, verify parts sum to the whole. The specification is comprehensive (OSO C17), not aggregated.

15.2 The 22 Categories

Pure (16 categories, no side effects, no grants required):

1. Text — 17 primitives for string construction, decomposition, search, and transformation.
2. Collections — 36 primitives for list construction, access, transformation, search, sorting, partitioning, aggregation, and combination.
3. VDR Arithmetic — 8 closed arithmetic, 5 active arithmetic, 3 structure ops, 10 comparison, 4 rounding, 7 extraction, 13 number theory, 8 list aggregates, 7 Q-basis, 8 functional remainder, 6 discrete calculus = 79 primitives covering the full VDR-1 through VDR-3 mathematical capability.

4. Sets — 14 primitives for construction, membership, algebra, and comparison.
5. Mappings — 15 primitives for dictionary construction, access, mutation, iteration, and combination.
6. Linear Algebra — 24 primitives for vector and matrix operations including determinant, inverse, solve, rank, power, and Gram-Schmidt.
7. Statistics and Probability — 16 primitives for descriptive statistics, probability operations, softmax, and distributions.
8. Conversion and Formatting — 14 primitives for type conversion, structured format parsing, and display formatting, plus 11 numeric conversion boundary operations.
9. Time — 10 primitives for date and time operations.
10. Identity — 8 primitives for hashing, encoding, and checksums.
11. Graphs — 13 primitives for graph algorithms.
12. Logic and Control — 11 primitives for conditional, iteration, error handling, type checking, and Prolog meta-operations.
13. KB Operations — 15 primitives for fact assertion, retraction, query, listing, scoping, and constraints.
14. Data Primitives — 53 primitives for LRU caches, counters, locks, queues, stacks, ring buffers, and bitsets.
15. Path and Mount — 17 primitives for dotted-path resolution, navigation, mounting, and connections.
16. Session Management — 8 primitives for snapshot, restore, reset, clone, kill, and comparison.
17. Domain Math — 8 polynomial, 4 finite field, 3 Markov, 2 graph math = 17 primitives for domain-specific mathematical operations.
18. Integer Fast Path — 13 integer operations plus 8 bit operations = 21 primitives for integer-only computation.
19. Denominator Management — 5 primitives for tracking and controlling denominator growth.

Operational (6 categories, side effects, require grants):

20. Filesystem — 15 primitives.
21. Compilation — 4 primitives.
22. Execution — 5 primitives.
23. Linting — 8 primitives.
24. Network — 5 primitives.
25. Process — 7 primitives.

15.3 Revised Counts

Type	Categories	Primitives
Pure (non-numeric)	11	207
Pure (numeric, VDR-1 through VDR-3 capability)	8	197
Operational	6	44
Total	**25**	**448**

Every primitive has an IOSE declaration. Every pure primitive is deterministic and bounded. Every operational primitive requires a positive credential grant and logs its execution. Every numeric primitive operates on exact VDR fractions, exact integers, or declared conversion boundaries with bounded error.

---

16. How the Three Components Integrate

The IOSE model defines the interfaces. The operational principles govern how the interfaces are used. The numeric builtins are the most complex set of interfaces.

When the LLM conducts an orchestrated inference (VDR-9), it issues command tokens that invoke builtins. Each builtin is an IOSE node — the command token parser validates the invocation against the IOSE declaration (type check), the constraint system checks operational principles (priority ordering, knowability thresholds, freshness requirements), and the execution produces results that enter the KB with provenance weights determined by the source's knowability rating.

When the lifecycle system (VDR-7) runs a training step, the forward pass invokes linear algebra builtins (exact matrix-vector products), the softmax builtin (exact probability distribution), and the loss computation (exact fraction). The denominator management builtins monitor growth and trigger reprojection when budgets are exceeded. Every value has a provenance chain. Every constraint is checked exactly. The IOSE model ensures the training pipeline's components connect correctly. The operational principles ensure data quality is prioritized over speed.

When a session is cloned (VDR-8) for a disposable worker, the IOSE declarations describe exactly what state is captured (live state: data primitives) and what persists independently (persistent: KB facts). The operational principle of idempotency ensures that if the clone re-asserts evidence facts, no duplicates are created. The force multiplier principle ensures the stable operator snapshot is verified before clones are launched from it.

The three components are not independent additions. They are the engineering foundation that makes the prior nine papers' specifications buildable.

---

17. Falsification Criteria

F1. If any IOSE declaration does not match the actual behavior of its component — the component produces an undeclared side effect, fails to produce a declared output, or accepts an input type not in its declaration — the IOSE model has a specification bug.

F2. If any operational principle encoded as a Prolog rule produces an incorrect decision when given correct inputs — the priority system recommends a lower-priority action when a higher-priority action is available — the principle encoding has a bug.

F3. If any numeric builtin produces an incorrect result from valid inputs — any of the VDR-1 through VDR-3 invariants are violated — the builtin implementation has a bug. 705 existing tests have not produced one such error.

F4. If a conversion boundary between number types produces a result whose error exceeds the declared bound, the conversion specification is wrong.

F5. If the type dispatch system selects a computation path that produces a different result than the canonical VDR path for the same inputs, the dispatch has a correctness bug.

F6. If the comprehensive builtin specification has a gap — an operation the LLM needs that is not covered by any of the 448 builtins or composable from them — the specification is incomplete. The slicing method should have caught it.

F7. If two categories in the builtin specification overlap — the same operation is defined in two categories with potentially different behavior — the specification has a consistency bug. The no-overlaps verification should have caught it.

---

18. Implementation Priority

Phase	Component	Dependencies	Effort
1	IOSE registry KB with declarations for all 448 builtins	VDR-5 KB struct	Medium
2	OSO principles KB (axioms, facts, rules, constraints)	Phase 1	Low
3	Number type hierarchy with dispatch	VDR-1 core	Medium
4	VDR closed arithmetic builtins	Phase 3	Low (already implemented)
5	VDR active arithmetic builtins	Phase 4	Low (already implemented)
6	Lift, rebase, comparison, rounding, extraction	Phase 4	Low (already implemented)

Phase	Component	Dependencies	Effort
7	Number theory builtins	Phase 3	Low
8	Q-basis operations	Phase 4, MATH-4	Medium
9	Functional remainder operations	Phase 4	Medium
10	Discrete calculus builtins	Phase 4	Low
11	Linear algebra builtins	Phase 4	Low (already implemented)
12	Probability and statistics builtins	Phase 11	Low (already implemented)
13	Domain-specific math (polynomial, finite field, Markov, graph)	Phases 4, 11	Medium
14	Integer fast path and bit operations	Phase 3	Low
15	Denominator management	Phase 4	Low
16	Conversion boundary operations	Phase 3	Low
17	IOSE validation (type checking, side effect preview, contract verification)	Phase 1	Medium
18	Integration testing: all 448 builtins against IOSE declarations	All above	High

Phases 4-6 and 11-12 are largely already implemented — VDR-1 through VDR-4 have working code for exact arithmetic, linear algebra, and the ML stack. The specification in this paper formalizes what exists as IOSE-declared builtins and extends it to cover capabilities that were tested in the gyms but not yet exposed as primitives.

---

19. Conclusion

Nine papers built a system with exact arithmetic, scoped knowledge, deterministic primitives, lifecycle management, runtime state, and structured inference. This paper provides the engineering foundation to build it: the IOSE system model for interfaces, operational principles for decisions, and comprehensive numeric builtins for mathematics.

The IOSE model gives every component a declared interface — inputs, outputs, side effects, properties. Components compose by connecting typed outputs to typed inputs.

Any component can be black-boxed. The system decomposes into verifiable subsystems. The declarations are Prolog facts in the root KB, queryable by the LLM and verifiable by the constraint system.

The operational principles give every decision a framework — the 90/9/0.9 priority system, the knowability spectrum, personal verification over hearsay, data primacy, comprehensive design, idempotency, one canonical method, operational vs application logic, control models vs understanding models, aged data, population statistics, and force multiplier safety. The principles are Prolog axioms, facts, rules, and constraints in the root KB, always in scope, enforceable.

The numeric builtins give the system the full mathematical capability of VDR-1 through VDR-3: exact closed and active arithmetic, lift and rebase, Q-basis transcendentals, functional remainder series, discrete calculus, linear algebra, probability and statistics, polynomial algebra, finite fields, Markov chains, denominator management, integer fast paths, and bit operations. 448 primitives across 25 categories, every one with an IOSE declaration.

The specification is the blueprint. The principles are the building code. The mathematics is the material. The system is buildable.

END HOWL-VDR-10-2026

Registry: [HOWL-VDR-10-2026]

Status: Specification complete

Domain: Applied Philosophy / Systems Architecture / Operational Engineering

Central Result: IOSE system model for all components, 15 operational engineering principles as enforceable Prolog rules, and 448 comprehensive builtins (404 pure, 44 operational) across 25 categories with full VDR-1 through VDR-3 mathematical capability.

Foundation: VDR-1 through VDR-9, MATH-3, MATH-4, Old School Operations

Key Principles: (1) Every component is an IOSE node. (2) Operational principles are enforceable Prolog facts, not suggestions. (3) The numeric stack exposes all tested VDR capabilities as builtins. (4) Comprehensive slicing ensures no gaps. (5) The specification is the blueprint for building the system.

Falsification: Seven criteria testable by IOSE contract verification, principle encoding testing, numeric invariant verification, conversion boundary checking, type dispatch correctness, gap detection, and overlap detection.

VDR-10 Extended Appendix Tables

Complete Reference Material for IOSE System Model, Operational Principles, and Comprehensive Numeric Stack

---

Appendix A: IOSE Declaration Registry — Complete Component List

A.1 Top-Level System Decomposition

Component	Category	Logic Type	Inputs	Outputs	Side Effects	Children
vdr llm prolog system	composite	operational	user prompt, context, active kbs	response text, kb mutations, direct data	env ops, grant consumption, session changes	All below
kb engine	composite	operational	op type, kb path, fact or query	query results, success	fact assert, fact retract, mutation log	scope resolver, fact store, rule engine
prolog engine	pure	operational	query, in scope kbs	bindings, success or fail	none	unifier, backtracker, cut handler
primitive executor	composite	operational	primitive id, resolved args	result	per primitive declared	pure dispatch, op dispatch, type checker
environment manager	composite	operational	env id, op, args, grant	exec result, exit code	process, file, network, grant dec, exec log	docker mgr, vm mgr, ssh mgr, local mgr

Component	Category	Logic Type	Inputs	Outputs	Side Effects	Children
session manager	composite	operational	operation, session name	session state	capture, restore, clear, clone, kill	snapshot engine, clone manager, reset handler
command token parser	pure	operational	raw token stream	parsed commands, text segments	none	tokenizer, path resolver, arg resolver
path registry	operational	operational	dotted path	integer id	id assigned if new	path store, slot registry
grant verifier	pure	operational	operation, location, user	authorized or denied	grant use decremented	grant store, hierarchy walker
constraint checker	pure	operational	constraint set	violations list	none	evaluator, reporter
inference orchestrator	composite	operational	notebook path, problem	conclusion, confidence	kb asserts, rules written, evidence stored	loop engine, mode selector, budget manager

A.2 IOSE Property Definitions

Property	Meaning	Verification Method	Applies To
pure	No side effects whatsoever	Run twice, compare system state	Pure primitives
deterministic	Same inputs always produce same outputs	Run N times, compare outputs	All pure, most operational
bounded	Terminates in time proportional to input size	Analysis or empirical bound	All pure (required)

Property	Meaning	Verification Method	Applies To
idempotent	f(f(x)) = f(x)	Apply twice, compare	session restore, bitset set, etc.
commutative	f(a,b) = f(b,a)	Swap args, compare	vdr add, vdr mul, set union
associative	f(f(a,b),c) = f(a,f(b,c))	Group differently, compare	vdr add, vdr mul, list concat
invertible	There exists g such that g(f(x)) = x	Apply inverse, compare	vdr neg, mat transpose
partial	May fail on some valid-typed inputs	Test failure cases	vdr div (zero), dict get (missing key)
lossless	No information lost in trans-formation	Roundtrip test	integer→fraction, cf roundtrip
lossy	Information may be lost	Compare with source	fraction→decimal display

A.3 IOSE Side Effect Taxonomy

Side Effect	Category	Reversible	Logged	Requires Grant
kb fact added	KB state	Yes (retract)	Yes	No (scoped by permissions)
kb fact removed	KB state	Yes (re-assert)	Yes	No (scoped by permissions)
mutation logged	KB metadata	No (append-only)	Inherent	No
scope changed	Context state	Yes (switch back)	Yes	No
constraint added	KB state	Yes (remove)	Yes	No
constraint removed	KB state	Yes (re-add)	Yes	No
constraint activated	KB state	Yes (suspend)	Yes	No
constraint suspended	KB state	Yes (activate)	Yes	No
primitive created	KB live state	Yes (destroy)	Yes	No

Side Effect	Category	Reversible	Logged	Requires Grant
counter mutated	KB live state	Depends (set=yes, inc=no)	Yes	No
lru entry added	KB live state	Yes (evict)	Yes	No
queue mutated	KB live state	No (pop is destructive)	Yes	No
stack mutated	KB live state	No (pop is destructive)	Yes	No
ring mutated	KB live state	No (overwrite is destructive)	Yes	No
lock state changed	KB live state	Yes (acquire/release)	Yes	No
bitset mutated	KB live state	Yes (set/clear)	Yes	No
live state captured	Session	No (read-only capture)	Yes	No
live state overwritten	Session	Yes (restore different)	Yes	No
live state cleared	Session	Yes (restore from snapshot)	Yes	No
clone created	Session	Yes (kill)	Yes	No
clone destroyed	Session	No	Yes	No
file accessed	Filesystem	No	Yes	Yes (read grant)
file created or overwritten	Filesystem	Partial (delete)	Yes	Yes (write grant)
file appended	Filesystem	No	Yes	Yes (write grant)
directory created	Filesystem	Yes (delete)	Yes	Yes (write grant)
file or directory removed	Filesystem	No	Yes	Yes (delete grant)
file moved	Filesystem	Yes (move back)	Yes	Yes (write grant)
file copied	Filesystem	Yes (delete copy)	Yes	Yes (write grant)
cpu used	Computation	No	Yes	Yes (compile/execute grant)
process created	Process	Yes (kill)	Yes	Yes (process grant)
process terminated	Process	No	Yes	Yes (process grant)
network request	Network	No	Yes	Yes (network grant)
grant use decremented	Authorization	No	Yes	Inherent
execution logged	Audit	No (append-only)	Inherent	No

Side Effect	Category	Reversible	Logged	Requires Grant
mount created	Path structure	Yes (unmount)	Yes	Conditional (read write needs grant)
mount removed	Path structure	Yes (re-mount)	Yes	No
connection added	KB structure	Yes (remove)	Yes	No
connection removed	KB structure	Yes (re-add)	Yes	No
id assigned	Path registry	No (permanent)	Yes	No

Appendix B: Operational Principle Cross-Reference Matrix

B.1 Principles Applied by System Phase

Principle	Prompt-Time	Inference (VDR-9)	Training (VDR-7)	Deployment	Monitoring	Session Mgmt
Control is foundation	✓ KB read before respond	✓ Evidence gathering	✓ Loss/gradient tracking	✓ Health checks	✓ Metric collection	✓ State capture
Knowability spectrum	✓ Source trust for answers	✓ Evidence confidence	✓ Data source ratings	✓ Metric reliability	✓ Staleness tracking	—
90/9/0.9 priorities	✓ Correctness over speed	✓ Exact over approximate	✓ Quality over speed	✓ Safety over throughput	✓ Alert priority	—
Personal vs hearsay	✓ Verify user claims	✓ Chain confidence	✓ Source verification	✓ Health check verify	✓ Cross-source verify	—
Data primacy	✓ KB facts over LLM memory	✓ Evidence over rules	✓ Data quality first	✓ Config as data	✓ Metrics as data	✓ State as data

Principle	Prompt-Time	Inference (VDR-9)	Training (VDR-7)	Deployment	Monitoring	Session Mgmt
Comprehensive	✓ Complete KB scope	✓ Evidence dimensions	✓ Corpus coverage	✓ Eval suite complete	✓ Metric coverage	✓ Full state capture
Idempotency	—	✓ Evidence re-assertion	✓ Checkpoint restore	✓ Rollback	—	✓ Session restore
One way to do it	✓ Canonical KB ops	✓ Canonical tool chain	✓ Canonical training loop	✓ Canonical deploy	✓ Canonical alerting	✓ Canonical snapshot
Ops vs app logic	✓ KB engine = ops	✓ Prolog = ops, Python = app	✓ Loop = ops, forward = app	✓ Serving = ops	✓ Collection = ops	✓ Manager = ops
Control vs understanding model	✓ KB over context window	✓ KB over LLM memory	✓ Checkpoint over estimate	✓ Config KB over assumption	✓ Prometheus over guess	✓ Snapshot over recall
Knowing the present	—	✓ Evidence freshness	✓ Metric lag awareness	✓ Health check lag	✓ All data aged	—
Population stats	—	✓ Confidence is population	✓ Denom growth is population	—	✓ Error rates are population	✓ Drift thresholds are population
Force multiplier	—	✓ Overconfidence check	✓ Verify before automate	✓ Canary before full	✓ Auto-rollback risk	✓ Verify snapshot before clone
Aggregated danger	✓ No orphan KBs	✓ Notebook has goal first	✓ Corpus categories first	✓ Eval before deploy	—	—
No logic in data store	✓ KB stores facts not procs	✓ Rules are data	✓ Config is data	✓ Deploy config is data	✓ Alert rules are data	—

B.2 Principle Conflict Resolution

Conflict	Principle A	Principle B	Resolution	Example
Speed vs correctness	Correctness (90)	Speed (0.9)	Correctness wins by 100x	Take extra query to verify rather than guess
Exact vs fast	Exact evidence (90)	Fast approximate (9)	Exact wins by 10x	Use VDR computation over LLM estimate
Comprehensive vs available	Define whole first (90)	Ship what exists (9)	Define whole, ship incrementally	Spec all 448 builtins, implement in phases
Verify vs deploy	Verify snapshot (90)	Deploy clone quickly (9)	Verify first	Run tests on stable operator before cloning
Data quality vs volume	Quality (90)	Quantity (9)	Quality wins by 10x	Filter low-quality sources even if corpus shrinks
Freshness vs availability	Fresh evidence (high)	Stale but available (low)	Refresh if budget allows, warn if not	Re-query Prometheus if data older than threshold
One way vs flexibility	Canonical method (high)	Custom approach (lower)	Canonical unless 10x advantage	Use kb assert not direct manipulation

B.3 Principle Encoding Statistics

Encoding Type	Count	Examples
Axioms (non-negotiable)	15	control is foundation, data primary over logic, all data is aged
Facts (knowability levels, source mappings)	~80	knowability level(fully knowable, 1/1, …), source knowability(prometheus live, controlled with lag)

Encoding Type	Count	Examples
Rules (decision procedures)	~60	evidence cascade, priority winner, should verify, arithmetic dispatch
Constraints (enforceable)	21	source must have knowability, no overconfident conclusions, one canonical method
Total Prolog terms in root.system.oso	**~176**	

Appendix C: Number Type Conversion Matrix

C.1 All Type-to-Type Conversions

From → To	Method	Lossless?	Error Bound	Builtin	Notes
integer → vdr fraction	[N, 1, 0]	Yes	0	vdr from integer	Free promotion
integer → qbasis	[N * 2^k, 2^k, 0]	Yes	0	implicit	Integer times shared denominator
integer → decimal display	str(N)	Yes	0	to string	Integers display exactly
vdr fraction → integer	floor/ceil/round/trunc	No (unless D=1)	up to 1	vdr floor etc.	Check vdr is integer first
vdr fraction → decimal display	long division to N digits	No	$\leq 5 \times 10^{(-N-1)}$	vdr to decimal string	Lossy display, source retained
vdr fraction → qbasis	round(V/D × 2^k)	No	$\leq 2^{(-k-1)}$	vdr reproject qbasis	Bounded, declared

From → To	Method	Lossless?	Error Bound	Builtin	Notes
vdr fraction → continued fraction	Euclidean algorithm	Yes (rationals)	0	vdr to continued fraction	Lossless for rationals
qbasis → vdr fraction	[p, 2^k, 0]	Yes	0	qbasis to fraction	qbasis IS a fraction
qbasis → integer	round(p / 2^k)	No	≤ 0.5	vdr round(qbasis to fraction(…))	Composed
qbasis → decimal display	via fraction	No	bounded by fraction→decimal	composed	Two-step
functional remainder → vdr fraction	resolve(fn, depth)	Yes (at depth)	0 (result is exact at that depth)	fn resolve	Not an approximation — exact at depth
decimal string → vdr fraction	parse “3.14” → [314, 100, 0]	Yes (terminating)	0 for terminating	vdr from decimal string	Conversion boundary entry point
ratio string → vdr fraction	parse “3/7” → [3, 7, 0]	Yes	0	vdr from ratio string	Exact
continued fraction → vdr fraction	evaluate CF	Yes	0	vdr from continued fraction	Exact
vdr fraction → egyptian	greedy algorithm	Yes	0 (sum recovers original)	vdr to egyptian	Lossless decomposition
vdr fraction → mixed	integer part + proper fraction	Yes	0	vdr to mixed	Lossless

C.2 Conversion Boundary Logging Format

```
ConversionBoundaryFact = struct {

source_type: Text,          // "decimal_string", "prometheus_float", etc.
```

```
target_type: Text,              // "vdr_fraction"
original_representation: Text,  // "4237.5" as string
converted_value: vdr_fraction,  // [8475, 2, 0]
method: Text,             // "decimal_string_to_fraction"
max_error: vdr_fraction,      // [0, 1, 0] for terminating decimals
turn: integer,
notebook: Text,            // inference notebook path if applicable
};
```

C.3 External Data Source to VDR Conversion Paths

External Source	Raw Format	Parse Step	Convert Step	Final VDR Type	Chain Length
Prometheus metric	JSON float string	parse json → dict get	vdr from decimal string	vdr fraction (closed)	6 links
REST API numeric	JSON number string	parse json → dict get	vdr from decimal string	vdr fraction (closed)	5 links
CSV cell	delimiter-separated string	parse csv → list nth	vdr from decimal string	vdr fraction (closed)	4 links
Database integer	integer	—	vdr from integer	vdr fraction [N,1,0]	2 links
Database decimal	decimal string	—	vdr from decimal string	vdr fraction (closed)	3 links
User typed “3/7”	ratio string	—	vdr from ratio string	vdr fraction [3,7,0]	2 links
User typed “0.5”	decimal string	—	vdr from decimal string	vdr fraction [1,2,0]	2 links
Q335 constant lookup	name string	—	qbasis get constant	qbasis [p, 2^335, 0]	1 link
Newton sqrt(2)	depth integer	—	fn sqrt(2, depth)	vdr fraction (exact at depth)	1 link

Appendix D: VDR Arithmetic Invariant Test Matrix

D.1 Closed Arithmetic Invariants

Invariant	Test Method	VDR-1 Tests	VDR-2 Tests	Total Passing	Expected
a + 0 = a	Random a, verify	10	20	30	30
a * 1 = a	Random a, verify	10	20	30	30
a + (-a) = 0	Random a, verify	10	20	30	30
a * (1/a) = 1	Random nonzero a, verify	10	20	30	30
a + b = b + a	Random a, b, verify	10	20	30	30
a * b = b * a	Random a, b, verify	10	20	30	30
(a+b)+c = a+(b+c)	Random a, b, c, verify	10	20	30	30
(a*b*)c = a(*b*c)	Random a, b, c, verify	10	20	30	30
a(*b+c*) = *a*b+a*c	Random a, b, c, verify	10	20	30	30
200-op roundtrip = 0 drift	Start 1/7, step 1/13	1	1	2	2
2000-op roundtrip = 0 drift	Start 1/7, step 1/13	—	1	1	1
Hilbert 3 × 3 inv exact	M*inv(M)=I	1	1	2	2
Hilbert 4 × 4 inv exact	M*inv(M)=I	1	1	2	2
Hilbert 5 × 5 inv exact	M*inv(M)=I	—	1	1	1
inv(inv(M)) = M	Random M	1	5	6	6
det(I) = 1	Various sizes	3	5	8	8

Invariant	Test Method	VDR-1 Tests	VDR-2 Tests	Total Passing	Expected
det(A*B)* = *det(A)*det(B)	Random A, B	—	5	5	5

D.2 Active Arithmetic Invariants

Invariant	Test Method	Verified In	Status
Active add preserves exact remainder structure	Construct known active, add, inspect R	VDR-1	Passing
Lift composition: lift(lift(R,a),b) = lift(R,a*b)	Random R, a, b	VDR-1	Passing
Lift identity: lift(R, 1) = R	Random R	VDR-1	Passing
Lift negation: lift(R, -1) = -R	Random R	VDR-1	Passing
Rebase preserves value: Pi(rebase(x, B)) = Pi(x)	Random x, B	VDR-1	Passing
Same-D rebase is identity	x with denominator D, rebase to D	VDR-1	Passing
Active neg: -(-x) = x	Random active x	VDR-1	Passing
Active add commutative	Random active a, b	VDR-1	Passing

D.3 Normalization Invariants

Invariant	Test Method	Verified	Status
Idempotent: normalize(normalize(x)) = normalize(x)	Random x of all types	VDR-1	Passing
Structural equality after normalization implies value equality	Random pairs	VDR-1	Passing
Value-equal objects normalize to structurally equal	[1,2,0] and [2,4,0]	VDR-1	Passing
Sign convention: D always positive after normalization	Random with negative D	VDR-1	Passing

Invariant	Test Method	Verified	Status
GCD reduction: gcd(V,D) = 1 for closed normalized	Random closed	VDR-1	Passing
No zero-sum children after normalization	Construct, normalize	VDR-1	Passing
Children sorted by denominator magnitude	Construct, normalize, inspect	VDR-1	Passing

---

Appendix E: VDR Gym Results Mapped to Builtins

E.1 Which Builtins Each Gym Domain Exercises

Gym	Domain	Primary Builtins Used	Secondary Builtins	Tests
01	Number theory	vdr gcd, vdr lcm, vdr mod, vdr euler totient, vdr harmonic sum	vdr add, vdr div, vdr binomial	37
02	Polynomial algebra	poly eval, poly add, poly mul, poly div, poly gcd, poly lagrange	vdr mul, vdr add, vdr pow	23
03	Continued fractions	vdr to continued fraction, vdr from continued fraction	vdr floor, vdr sub, vdr reciprocal	31
04	Matrix decomposition	vdr mat mul, vdr mat inv, vdr mat det, vdr mat solve, vdr mat pow	vdr vec dot, vdr mat transpose	13
05	Recursive sequences	vdr fibonacci, vdr add, vdr mul, vdr pow	vdr sub, vdr binomial	15
06	Combinatorics	vdr binomial, vdr factorial, vdr sum	vdr mul, vdr div, int pow	31
07	Signal processing	vdr dot product, vdr mat matvec, vdr sum	vdr mul, vdr add	11
08	Computational geometry	vdr vec sub, vdr vec dot, vdr mat det	vdr div, vdr compare	19
09	Differential equations	vdr discrete derivative, vdr left riemann, vdr trapezoidal	vdr add, vdr mul, vdr div, fn exp	10
10	Optimization	fn sqrt, vdr discrete derivative	vdr sub, vdr div, vdr compare	8

Gym	Domain	Primary Builtins Used	Secondary Builtins	Tests
11	Probability	vdr prob bayes, vdr prob normalize, vdr prob expected, markov steady state	vdr mat solve, vdr sum, vdr binomial	13
12	Cryptographic primitives	vdr mod pow, vdr mod inv, vdr extended gcd, vdr chinese remainder	gf add, gf mul, gf inv	37
13	Symbolic algebra	poly derivative, poly integral, poly eval, vdr sum	vdr pow, vdr mul, vdr div	20
14	Fixed-point iteration	fn sqrt, vdr compare, vdr abs	vdr sub, vdr div	partial
15	Chaos and sensitivity	vdr mul, vdr mod, vdr abs	vdr sub, vdr denominator	partial
16	Graph theory	vdr mat mul, vdr mat pow, pagerank exact, vdr compare	vdr add, vdr min	20
17	Game theory	vdr mat solve, vdr prob normalize	vdr sub, vdr div, vdr compare	24
18	Coding theory	gf add, gf mul, gf inv, gf pow	vdr mod, int mod	27
19	Algebraic topology	vdr mat mul, vdr mat rank	vdr mat add, vdr mat scale	16
20	Tropical/lattice	vdr min, vdr mat mul (min-plus), vdr mat gram schmidt	vdr vec dot, vdr compare	23
21	Control theory	vdr mat pow, vdr mat det, vdr mat mul	vdr mat matvec, poly eval	13
22	Wavelets	vdr vec add, vdr vec sub, vdr vec scale, vdr mat mul	vdr mat inv, vdr dot product	18
23	Q335 transcendentals	qbasis add, qbasis sub, qbasis mul, qbasis scalar mul	vdr add, vdr mul, fn exp	16

E.2 Builtin Coverage by Gym Tests

Builtin Category	Gyms Exercising	Total Tests	Coverage Level
VDR closed arithmetic	All 23	507	Complete
VDR active arithmetic	01, 05, 14, 15	~50	Moderate
Lift and rebase	03, 14	~20	Light
Comparison	All 23	507	Complete
Number theory	01, 06, 12	105	Heavy

Builtin Category	Gyms Exercising	Total Tests	Coverage Level
Q-basis	23	16	Light
Functional remainder	09, 10, 14	~25	Moderate
Discrete calculus	09	10	Light
Linear algebra	04, 07, 08, 16-22	~150	Heavy
Probability/statistics	11, 17	37	Moderate
Polynomial	02, 13, 21	~55	Moderate
Finite field	18	27	Focused
Markov	11, 21	~15	Light
Integer fast path	01, 06, 12, 18	~80	Moderate

Appendix F: Denominator Growth Reference

F.1 Growth by Operation Type

Operation	Input Denom Sizes	Output Denom Size	Growth Factor	Notes
Closed addition (a/b + c/d)	b, d	$b \times d$ (before GCD)	Up to $b \times d$	GCD reduction may shrink
Closed multiplication (a/b $\times$ c/d)	b, d	$b \times d$ (before GCD)	Up to $b \times d$	GCD reduction may shrink
Closed division (a/b $\div$ c/d)	b, d	$b \times c$ (before GCD)	Up to $b \times c$	Uses numerator of divisor
Active addition (same D)	D	D	1 (no growth)	Best case
Active addition (diff D)	D_1, D_2	$D_1 \times D_2$	Up to $D_1 \times D_2$	Frame expansion
Active multiplication	D_1, D_2	$D_1 \times D_2$	Up to $D_1 \times D_2$	Plus remainder growth

Operation	Input Denom Sizes	Output Denom Size	Growth Factor	Notes
Matrix multiplication ($n \times n$)	D max	D max^(2n) worst case	Exponential in dimension	GCD helps in practice
Softmax (Taylor depth N)	D logits	(N!)^len $\times$ D logits	Very large	Q-basis reprojection recommended
SGD update (1 step)	D param, D lr, D grad	D param $\times$ D lr $\times$ D grad	Triple product	Reprojection at checkpoints

F.2 Denominator Growth During Training (Empirical Estimates)

Training Phase	Steps	Typical Denom Bits	After Reprojection	Notes
Initialization (Xavier)	0	10-15	—	Small from initialization
Early training	1-1000	15-25	—	Linear growth
Mid training	1000-10000	25-40	35 (at checkpoint)	Reprojection resets
Late training	10000-50000	35-50	35 (at each checkpoint)	Stable with reprojection
Fine-tuning (small LR)	+5000	25-35	—	Slow growth
RLHF/DPO	variable	30-45	monitor closely	Reward signal can spike growth

F.3 Reprojection Error Bounds

Q-basis Exponent K	Precision (decimal digits)	Error Bound	Relative to Planck Length	Sufficient For
53	~16	2^(-54) ≈ 5.6 × 10^(-17)	10^(-18) larger than Planck	Float64 equivalent
100	~30	2^(-101) ≈ 3.9 × 10^(-31)	10^4 smaller than Planck	Scientific computation
200	~60	2^(-201)	10^34 smaller than Planck	High-precision research
335	~100	2^(-336)	10^66 smaller than Planck	Q335 standard basis
668	~200	2^(-669)	10^168 smaller than Planck	Q335 multiplication products

---

Appendix G: Builtin Dependency Graph

G.1 Which Builtins Depend on Which

Builtin	Depends On	Used By
vdr add	(foundational)	vdr sum, vdr mean, stat variance, vec add, mat add, poly add, discrete derivative, left riemann
vdr mul	(foundational)	vdr product, vdr pow, vec scale, vec dot, mat mul, mat scale, poly mul, softmax, prob joint
vdr div	(foundational)	vdr mean, vdr reciprocal, prob bayes, prob normalize, discrete derivative, stat variance
vdr neg	(foundational)	vdr sub, vec neg, mat subtraction
vdr sub	vdr add, vdr neg	stat variance, vec sub, discrete derivative

Builtin	Depends On	Used By
vdr pow	vdr mul	fn exp (Taylor), poly eval (Horner), vdr mod pow, mat pow
vdr reciprocal	vdr div	active div by closed, prob normalize
vdr gcd	(foundational integer)	vdr simplify, vdr lcm, vdr extended gcd, vdr mod inv
vdr simplify	vdr gcd	All arithmetic (normalization)
vdr compare	vdr mul (cross-multiply)	vdr min, vdr max, list sort (on fractions), vdr less than
vdr lift	(structure op)	active add diff d, vdr rebase
vdr rebase	vdr lift, vdr mod	vdr reproject qbasis
fn resolve	(functional op)	fn sqrt, fn exp, fn log, fn sin, fn cos, vdr scalar projection (on functional R)
fn exp	vdr pow, vdr div, vdr factorial	vdr softmax, prob entropy terms
vdr mat det	vdr mul, vdr sub	vdr mat inv, vdr mat solve, vdr mat rank
vdr mat inv	vdr mat det	vdr mat solve (alternative), vdr mat gram schmidt
vdr mat mul	vdr vec dot	vdr mat pow, adjacency matrix power, markov n steps
vdr mat solve	vdr mat det	markov steady state, pagerank exact
poly eval	vdr mul, vdr add	poly lagrange (evaluation check)
vdr prob normalize	vdr sum, vdr div	vdr softmax, vdr prob conditional, markov steady state
vdr softmax	fn exp, vdr prob normalize	VDR-4 transformer forward pass
qbasis add	int add	(standalone for same-exponent)
qbasis mul	int mul, bit shift right	(requires reprojection for result)

G.2 Foundational Builtins (No Dependencies)

Builtin	Category	Why Foundational
vdr add	closed arithmetic	All arithmetic builds on addition
vdr mul	closed arithmetic	Multiplication is independent of addition
vdr neg	closed arithmetic	Negation is structural
vdr gcd	number theory	GCD is the normalization engine
vdr compare	comparison	Ordering via cross-multiplication
vdr lift	structure	Remainder transport
int add	integer fast path	Integer addition
int mul	integer fast path	Integer multiplication
bit and	bit ops	Bitwise foundational

These must be implemented first. Everything else composes from them.

Appendix H: IOSE Validation Test Plan

H.1 Type Compatibility Tests

Test	Chain	Check	Expected
Integer→VDR arithmetic	int add → vdr add	Output type of int add (integer) promotes to vdr fraction input	Pass (lossless promotion)
VDR→list sort	vdr add → list sort	List of vdr fractions as input to list sort	Pass (list(vdr fraction) is list)
String→VDR	string split → vdr from decimal string	String atoms to fraction conversion	Pass (atom → vdr fraction)
Incompatible	vdr add → string length	VDR fraction output → string input	Fail (type mismatch detected)
Incompatible	lock check → vdr mat mul	Boolean output → matrix input	Fail (type mismatch detected)
JSON→VDR chain	parse json → dict get → vdr from decimal string	Full Prometheus integration path	Pass
Multi-step inference	net fetch → parse json → list map(to fraction) → stat mean → kb assert	Full evidence pipeline	Pass

H.2 Side Effect Preview Tests

Chain	Expected Side Effects	Grant Required
[kb assert, kb assert, counter inc]	[kb fact added × 2, mutation logged × 2, counter mutated × 1]	No
[net fetch, parse json, vdr from decimal string]	[network request × 1]	Yes (network)
[fs read, parse csv, list sort]	[file accessed × 1]	Yes (filesystem read)
[env exec, store result, kb assert]	[arbitrary side effects × 1, kb fact added × 1]	Yes (execute)
[vdr add, vdr mul, vdr compare]	[] (empty — all pure)	No

H.3 Contract Verification Tests

Component	Declared Output	Actual Output	Declared SE	Actual SE	Verdict
vdr add([1,2,0], [1,3,0])	vdr fraction	[5,6,0]	[]	[]	Contract satisfied
kb assert(fact)	void	void	[kb fact added]	[kb fact added, mutation logged]	mutation logged is sub-effect of kb fact added — OK
net fetch(url) without grant	exec result	—	[network request]	denied before execution	Grant check prevents execution — correct

Appendix I: Numeric Builtin Count Reconciliation

I.1 From VDR-6 to VDR-10

Category	VDR-6 Count	VDR-10 Count	Delta	New Builtins Added
Basic arithmetic	26	8 closed + 5 active + 3 structure = 16	-10	Active arithmetic, lift, rebase, projection split out
Comparison	(included in 26)	10	+10	Dedicated comparison category
Rounding/extraction	(included in 26)	11	+11	Dedicated rounding, extraction, state queries
Number theory	(included in 26)	13	+13	gcd, lcm, mod, mod pow, mod inv, ext gcd, prime test, totient, CRT
List aggregates	(included in 26)	8	+8	sum, product, mean, weighted sum, dot, sum sq, harmonic, alternating
Q-basis	0	7	+7	Full MATH-4 constant operations
Functional remainder	0	8	+8	sqrt, exp, log, sin, cos, resolve, make newton, make series
Discrete calculus	0	6	+6	derivative, nth derivative, Riemann, trapezoidal, finite diff, Richardson
Linear algebra	16	24	+8	vec new, vec dim, vec get, vec neg, mat new, mat dims, mat get, mat pow, gram schmidt
Statistics/probability	16	16	0	Unchanged
Conversion boundaries	0	11	+11	from integer, from decimal, from ratio, to decimal, to percentage, to scientific, to mixed, to/from CF, to egyptian, format fraction
Polynomial	0	8	+8	eval, add, mul, div, gcd, derivative, integral, Lagrange

Category	VDR-6 Count	VDR-10 Count	Delta	New Builtins Added
Finite field	0	4	+4	GF(p) add, mul, inv, pow
Markov	0	3	+3	steady state, step, n steps
Graph math	0	2	+2	adjacency power, pagerank exact
Integer fast path	0	13	+13	Fast integer ops bypassing denominator
Bit operations	0	8	+8	AND, OR, XOR, NOT, shifts, popcount, bit width
Denominator management	0	5	+5	denom bits, denom digits, reproject, budget check, precision state
Numeric total	**58**	**173**	**+115**	

I.2 Full System Builtin Count

Category Group	VDR-6 Count	VDR-10 Count	Source
Text	17	17	Unchanged from VDR-6
Collections	34→36	36	Minor additions in VDR-8
Numeric (all)	58	173	This paper
Sets	14	14	Unchanged
Mappings	15	15	Unchanged
Conversion/formatting (non-numeric)	14	14	Unchanged
Time	10	10	Unchanged
Identity	8	8	Unchanged
Graphs	13	13	Unchanged
Logic/Control	11	11	Unchanged
KB Operations	15	15	Unchanged
Data Primitives	53	53	From VDR-8
Path/Mount	17	17	From VDR-8
Session	8	8	From VDR-8
Filesystem	15	15	Unchanged
Compilation	4	4	Unchanged
Execution	5	5	Unchanged
Linting	8	8	Unchanged
Network	5	5	Unchanged
Process	7	7	Unchanged
Grand total	**333**	**448**	**+115 numeric**

I.3 Primitive Classification Summary

Classification	Count	Percentage
Pure, deterministic, bounded	392	87.5%
Pure, deterministic, partial (may fail on some inputs)	12	2.7%
Operational (requires grant)	44	9.8%
Total	**448**	**100%**
With idempotent property	47	10.5%
With commutative property	38	8.5%
With associative property	24	5.4%
With invertible property	14	3.1%
With lossless property	11	2.5%

Appendix J: OSO Concept to VDR-LLM-Prolog Mapping

J.1 Complete Concept Mapping

OSO Concept ID	OSO Name	VDR System Manifestation	Implemented In
C1	Operations	System lifecycle management	VDR-7
C2	Control	KB observation + primitive agency	VDR-5, VDR-6
C3	Automation	Disposable clones, inference loop	VDR-8, VDR-9
C5	Real	External world (Prometheus, APIs, files)	VDR-6 operational
C6	Virtual	KB facts, VDR fractions, Prolog rules	VDR-5, VDR-1
C7	Knowability	Source confidence spectrum	This paper §3.3
C8	Data	KB facts — fully knowable	VDR-5
C9	Logic	Prolog rules, Python scripts — partially knowable	VDR-5, VDR-6
C11	System	The VDR-LLM-Prolog system as IOSE network	This paper §2
C12	Universal Machine	IOSE node model	This paper §2.2
C13	IOSE	Input/Output/Side-Effect interface	This paper §2.2
C14	Systemic Thinking	IOSE network modeling	This paper §2.3
C15	Philosopher's Knife	Slicing the Pie for builtin categories	This paper §15

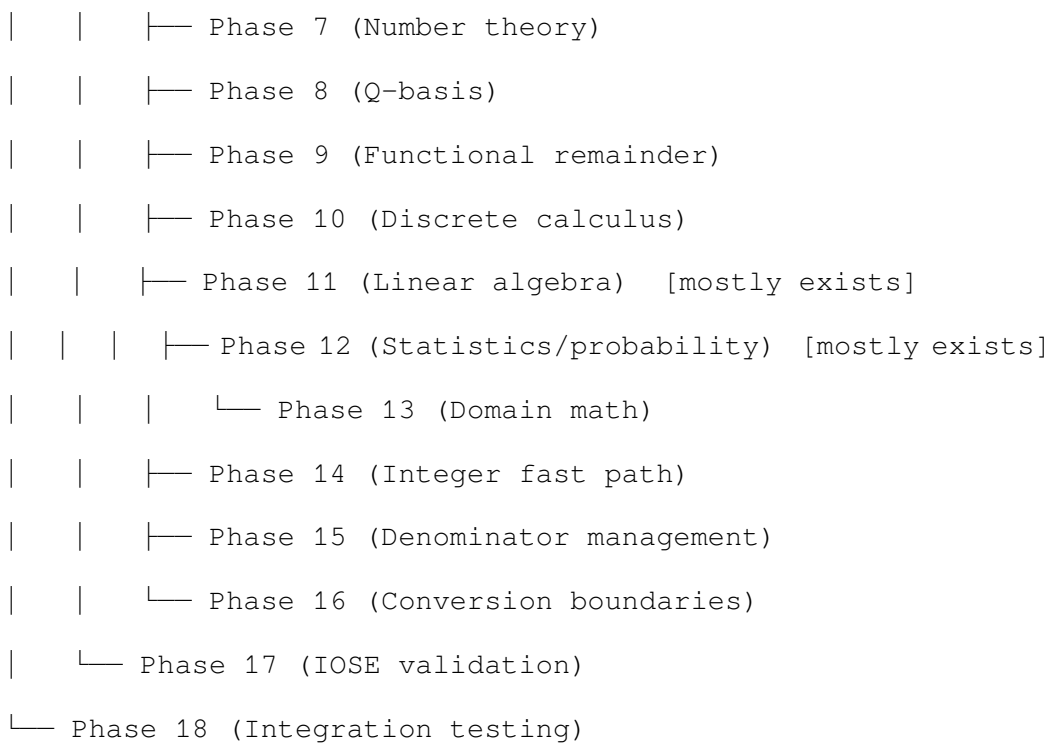
OSO Concept ID	OSO Name	VDR System Manifestation	Implemented In
C16	Slicing the Pie	25 categories covering the whole space	This paper §15
C17	Comprehensive System	Top-down spec, no gaps, no overlaps	This paper §3.7
C18	Aggregated System	What to avoid during implementation	This paper §3.7
C19	Engineering	Efficient use of resources for desired effect	All papers
C20	Attribute Axis	Knowability, priority, freshness axes	This paper §3
C21	Axiomatic Engineering	90/9/0.9 priority system	This paper §3.4
C22	Alignment	Evaluation-based, requires human judgment	This paper §3.14
C23	Internal Consistency	Constraint-based, automated	This paper §3.14
C24	Impersonal Decision Making	Priority system produces same decision regardless of decider	This paper §3.4
C25	Best (anti)	Avoided — system uses explicit tradeoffs	This paper §3.4
C26	Better	Explicit comparison with priorities	This paper §3.4
C29	Production Environment	Deployment KB from VDR-7	VDR-7 Phase 9
C31	Operational Logic	KB engine, primitive executor, session manager	This paper §3.10
C32	Application Logic	User Python scripts, LLM-generated content	This paper §3.10
C33	One Way To Do It	One canonical method per task	This paper §3.9
C35	DOS	KB tree as unified system	VDR-5
C36	Idempotency	Tagged on every operation	This paper §3.8
C37	Modeling	Two model types distinguished	This paper §3.11
C39	Model for Control	KB, data primitives, snapshots	This paper §3.11
C40	Source of Truth	KB engine, Prometheus, deployment KB	This paper §3.11
C41	Knowing the Present	All data is aged; freshness tracking	This paper §3.12

OSO Concept ID	OSO Name	VDR System Manifestation	Implemented In
C42	Black-Boxing	IOSE composite nodes	This paper §2.4
C47	Personal Experience	VDR computation, KB verification	This paper §3.5
C48	Hearsay	External data, user claims, LLM generation	This paper §3.5
C54	Logic in Data Store (anti)	No stored procedures in KB	This paper §3.6
C55	Magical Thinking (anti)	Provenance chain prevents "it just happened"	VDR-9
C66	Force Multiplier	Automation amplifies both fixes and failures	This paper §3.14
R1	90/9/0.9	Priority system for all decisions	This paper §3.4
R2	0-1-Infinity	Architecture scaling rule	Applied in VDR-7
R3	Idempotency	Safe re-run of operations	This paper §3.8
R8	Data Primacy	KB facts over logic	This paper §3.6
R12	Verify Personally	Check before trusting hearsay	This paper §3.5
R16	Population Stats Only	Confidence is population prediction	This paper §3.13

Appendix K: Implementation Phase Dependencies

K.1 Phase Dependency Graph

```
Phase 1 (IOSE registry)
├── Phase 2 (OSO principles KB)
├── Phase 3 (Number type hierarchy)
│   ├── Phase 4 (Closed arithmetic)  [mostly exists]
│   │   ├── Phase 5 (Active arithmetic)  [mostly exists]
│   │   ├── Phase 6 (Lift, rebase, comparison)  [mostly exists]
```



K.2 What Already Exists vs What Needs Building

Phase	Status	Existing Code	New Work Needed
1	New	—	IOSE registry KB, declaration format, query interface
2	New	—	Prolog encoding of all OSO principles
3	New	VDR type exists	Dispatch layer, promotion rules, boundary logging
4	Exists	vdr.py	Wrap as IOSE-declared builtins
5	Exists	active mul.py	Wrap as IOSE-declared builtins
6	Exists	vdr.py (lift, rebase)	Wrap as IOSE-declared builtins
7	Partial	GCD in vdr.py	mod pow, mod inv, ext gcd, CRT, primality, totient

Phase	Status	Existing Code	New Work Needed
8	Partial	basis.py (Q335)	Full IOSE-declared operations with error tracking
9	Exists	fn.py	Wrap as IOSE-declared builtins, add sin/cos
10	Exists	fn.py (discrete calc)	Wrap as IOSE-declared builtins, add Richardson
11	Exists	linalg.py	Add mat pow, gram schmidt, wrap as IOSE
12	Exists	softmax.py + ML stack	Wrap stat builtins as IOSE
13	Partial	Gym test code	Extract into reusable builtins
14	New	—	Integer fast-path dispatch
15	Partial	Denominator tracking in trainer	Formalize as builtins
16	New	—	Conversion boundary logging
17	New	—	Type checker, SE previewer, contract verifier
18	New	705 existing tests	New IOSE contract tests

Appendix L: Cumulative System Statistics

L.1 Complete Paper Series

Paper	Registry	Central Result
VDR-1	[HOWL-VDR-1-2026]	Exact arithmetic in irreducible triple form
VDR-2	[HOWL-VDR-2-2026]	15 domains, 282 tests
VDR-3	[HOWL-VDR-3-2026]	23 domains, transcendental integration
VDR-4	[HOWL-VDR-4-2026]	24-module ML stack, working exact transformer
VDR-5	[HOWL-VDR-5-2026]	Prolog KB architecture, constraints, scoped knowledge
VDR-6	[HOWL-VDR-6-2026]	255 primitives, command tokens, operational environments
VDR-7	[HOWL-VDR-7-2026]	12-phase lifecycle, training through retirement
VDR-8	[HOWL-VDR-8-2026]	Data primitives, dotted paths, session management
VDR-9	[HOWL-VDR-9-2026]	Orchestrated Inference: structured reasoning through tool composition

Paper	Registry	Central Result
VDR-10	**[HOWL-VDR-10-2026]**	**IOSE system model, operational principles, 448 comprehensive builtins**

L.2 System Capability Progression

Paper	What It Added	Cumulative Capability
VDR-1–4	Exact arithmetic + ML stack	Can compute exactly
VDR-5	Knowledge bases, constraints, scoping	Can know and constrain
VDR-6	Primitives, commands, environments	Can do and execute
VDR-7	Lifecycle management	Can train, deploy, and retire
VDR-8	Runtime state, addressing, sessions	Can remember, address, and recover
VDR-9	Orchestrated Inference	Can investigate, reason, and conclude
VDR-10	**IOSE model, engineering principles, full math**	**Can be built, tested, and operated**

L.3 Complete Module Count

Layer	Count	Source
Arithmetic	5	VDR-1
Transcendental	3	VDR-4
ML	4	VDR-4
Infrastructure	8	VDR-4
Architecture	4	VDR-4
Logic/Knowledge	3	VDR-5
Execution	3	VDR-6
Lifecycle	4	VDR-7
Runtime state	3	VDR-8
IOSE and principles	3 (new)	VDR-10
Total	**40**	

New VDR-10 modules: `iose_registry.py` (IOSE declaration storage, validation, query), `oso_principles.py` (operational engineering KB loading), `numeric_dispatch.py` (type hierarchy, automatic promotion, fast-path selection).

L.4 Complete Builtin Count

Type	Count	Source
Pure (non-numeric)	207	VDR-6, VDR-8
Pure (numeric)	197	VDR-10 (replaces VDR-6's 58)
Operational	44	VDR-6
Total	**448**	

L.5 Complete Test Count

Source	Existing	Planned	Total
VDR-1 through VDR-4	705	—	705
VDR-6 non-numeric pure	—	615	615
VDR-6 operational	—	132	132
VDR-7 lifecycle	—	200	200
VDR-8 data primitives	—	129	129
VDR-8 path/session	—	75	75
VDR-9 inference	—	100	100
VDR-10 numeric builtins (3 per builtin $\times$ 173)	—	519	519
VDR-10 IOSE validation	—	80	80
VDR-10 OSO principle encoding	—	50	50
VDR-10 integration	—	60	60
Total	**705**	**1960**	**2665**

L.6 KB Struct Fields (Final)

Field	Source	Classification
name	VDR-5	Identity
path	VDR-8	Identity
id	VDR-8	Identity
facts	VDR-5	Persistent
rules	VDR-5	Persistent

Field	Source	Classification
constraints	VDR-5 addendum	Persistent
connections	VDR-8	Persistent
working data	VDR-5	Live
lrus	VDR-8	Live
counters	VDR-8	Live
locks	VDR-8	Live
queues	VDR-8	Live
stacks	VDR-8	Live
buffers	VDR-8	Live
bitsets	VDR-8	Live
parent id	VDR-5/8	Structural
children ids	VDR-5/8	Structural
mounts	VDR-8	Structural
visibility	VDR-5	Metadata
frozen	VDR-5	Metadata
owner	VDR-5 addendum	Metadata
created at	VDR-5	Metadata
last modified	VDR-5	Metadata
iose declaration	VDR-10	Metadata

24 fields + 1 new IOSE declaration field = **25 fields total**.

END VDR-10 EXTENDED APPENDIX TABLES

[**HOWL-VDR-10-2026**] Operational Foundations and Comprehensive Builtin Specification. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-10-2026

[**HOWL-VDR-1-2026**] VDR Arithmetic: Value, Decimal, Remainder. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-1-2026

[**HOWL-VDR-2-2026**] VDR Gym: Exact Arithmetic Across Fifteen Domains. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-2-2026

[**HOWL-MATH-3-2026**] The Transcendental Hierarchy. Github: https://github.com/ghowland/papers/tree/main/papers/M
MATH-3-2026

[**HOWL-MATH-4-2026**] The Universal Power-of-Two Basis. Github: https://github.com/ghowland/papers/tree/main/pape
MATH-4-2026

[**HOWL-VDR-3-2026**] VDR Gym Extension. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL
VDR-3-2026

[**HOWL-VDR-4-2026**] Exact-Fraction Language Model Architecture. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-4-2026

[**HOWL-LLM-1-2026**] Integer LLM with Prolog Knowledge Base. Github: https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-1-2026

[**HOWL-VDR-5-2026**] Exact Arithmetic Meets Logical Provenance. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-5-2026

[**HOWL-VDR-6-2026**] Computational Primitives and Operational Environments. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-6-2026

[**HOWL-VDR-7-2026**] Complete Lifecycle Technical Specification. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-7-2026

[**HOWL-VDR-8-2026**] Computational State Primitives, Universal Data Pathing, and Session Management. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-8-2026

[**HOWL-VDR-9-2026**] Orchestrated Inference. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOW
VDR-9-2026